

|                  |                                                                                       |
|------------------|---------------------------------------------------------------------------------------|
| Experiment No.   | 1                                                                                     |
| Experiment Title | <a href="#">Working with Python Packages – NumPy, SciPy, Scikit-learn, Matplotlib</a> |
| Student Name     | Gunaseelan R                                                                          |
| Register Number  | 9342507779                                                                            |
| Faculty          | Dr. Poreddy Ajay Kumar Reddy                                                          |
| Submission Date  | 16.08.2026                                                                            |

## 1 Aim

To explore the core Python data-science stack (NumPy, Pandas, SciPy, Scikit-learn, Matplotlib), and to build reusable exploratory-data-analysis (EDA) tooling that can be pointed at different *kinds* of datasets – plain tabular, text, and image – as a way of understanding what an ML workflow actually looks like before picking a model.

**A note on scope:** this write-up covers exactly what was implemented and run – two dataset explorations and one model-comparison smoke test. It doesn't claim coverage of every dataset listed in the manual – the Handwritten Character/MNIST image EDA wasn't run in this pass, and Predicting Diabetes was only smoke-tested via Scikit-learn's built-in toy dataset (Section 5).

## 2 Approach

Rather than writing one-off EDA code per dataset, the work here is organized around three **reusable, generic EDA functions** – one for numeric/tabular data, one for text data, and one for image folders – each producing a single 12-subplot diagnostic figure covering things like missing values, distributions, correlations, outliers and class balance in one shot. A fourth block builds a small generic harness that trains several regression *or* classification models on whatever (`X_train`, `X_test`, `y_train`, `y_test`) it's handed and prints out standard metrics for each.

### Libraries and functions actually used

| Library | Functions / modules used                                                                                                                                 |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| NumPy   | array ops, <code>argsort</code> , <code>asarray</code> , boolean masking, <code>np.number</code> dtype selection                                         |
| Pandas  | <code>read_csv</code> , <code>describe</code> , <code>isnull().sum()</code> , <code>value_counts</code> , <code>select_dtypes</code> , <code>corr</code> |
| SciPy   | <code>scipy.stats.probplot</code> (for the Q-Q normality plot)                                                                                           |

---

|              |                                                                                                                                                                                                                                                                                                                                                                         |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Scikit-learn | CountVectorizer; LinearRegression, Ridge, Lasso, DecisionTreeRegressor/Classifier, RandomForestRegressor/Classifier, SVR, SVC, KNeighborsRegressor/Classifier, GaussianNB, LogisticRegression; mean_absolute_error, mean_squared_error, r2_score, accuracy_score, precision_score, recall_score, f1_score, confusion_matrix; load_iris, load_diabetes, train_test_split |
| Matplotlib   | subplots, hist, boxplot, violinplot, scatter, imshow, bar, barh, custom rcParams styling (serif font, bold axis labels)                                                                                                                                                                                                                                                 |
| Pillow (PIL) | Image.open, resizing and mode detection for the image EDA function                                                                                                                                                                                                                                                                                                      |

---

### 3 Dataset 1: Loan Approval – Numeric EDA

#### Code Summary

`eda_summary_figure()` takes any dataframe and an optional target column, prints shape/missing-values/summary-stats/target-distribution, then lays out a 3×4 grid: missing-value bar, correlation heatmap, histogram, KDE, boxplot, Q-Q plot, multi-feature outlier boxplot, scatter, violin plot, class-balance bar, per-feature variance bar, and a second histogram.

#### Actual Output

---

|                     |                                |
|---------------------|--------------------------------|
| Shape               | 4269 rows × 13 columns         |
| Missing values      | 0 (none found in any column)   |
| Target column used  | <code>loan_status</code>       |
| Target distribution | Approved: 2656, Rejected: 1613 |

---

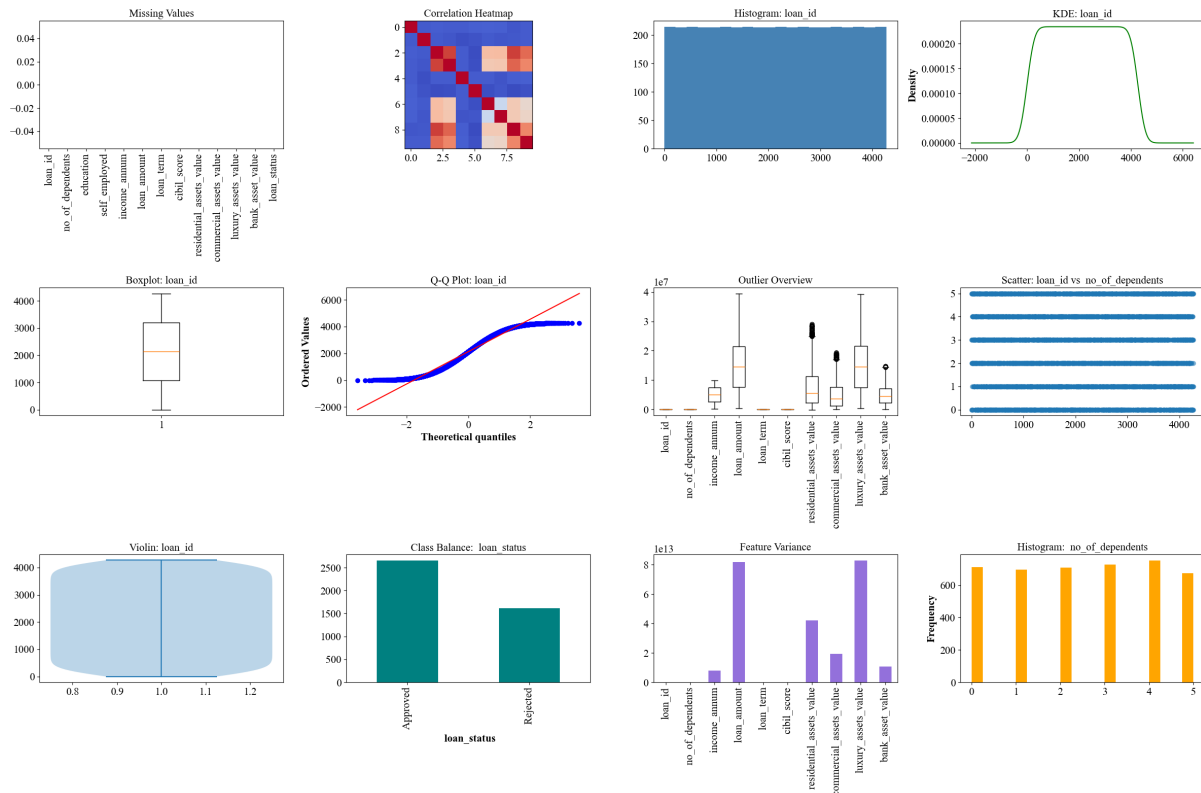


Figure 1: Generated 12-panel EDA figure – Loan Approval Dataset (actual run output)

### Inference:

The run completed cleanly – zero missing values across all 4269 rows is a genuinely clean dataset to start with. One honest limitation worth calling out: the function grabs "the first two numeric columns" (`c1`, `c2`) for the histogram/KDE/boxplot/scatter/violin panels without checking whether they're actually meaningful, and in this dataset those happen to be `loan_id` and `no_of_dependents` – so half the 12 panels are analyzing a plain row-index column, which is not useful information (that's exactly why the histogram of `loan_id` is a flat uniform bar and the KDE is just a boxy plateau). The genuinely useful panels here are the correlation heatmap (visible red block among the asset-value columns, suggesting they move together) and the `loan_status` class-balance bar, which shows a moderate 62/38 imbalance towards approvals. **A naming note:** the manual calls this dataset "Loan Amount Prediction" (implying a regression target on the loan amount), but the code as written uses `loan_status` (Approved/Rejected) as the target – so what was actually explored here is set up as a *classification* problem, not a regression one. That's a genuine mismatch worth flagging rather than papering over.

## 4 Dataset 2: Email Spam – Text/NLP EDA

### Code Summary

`eda_summary_figure_nlp()` takes a text column and optional target, and builds a similar 12-panel grid but tailored to text: class distribution, character-length and word-count histograms, length-by-class boxplot, top-15 word frequencies overall and per class (via `CountVectorizer` with English stopwords removed), average word length, unique-word

ratio (vocabulary richness), punctuation count, top-15 bigrams, and a word-count-vs-length scatter.

## Actual Output

---

|                       |                                    |
|-----------------------|------------------------------------|
| Documents             | 5728                               |
| Avg. character length | 1556.8                             |
| Avg. word count       | 326.8                              |
| Missing text entries  | 0                                  |
| Target column used    | <b>spam</b>                        |
| Target distribution   | Not spam (0): 4360, Spam (1): 1368 |

---

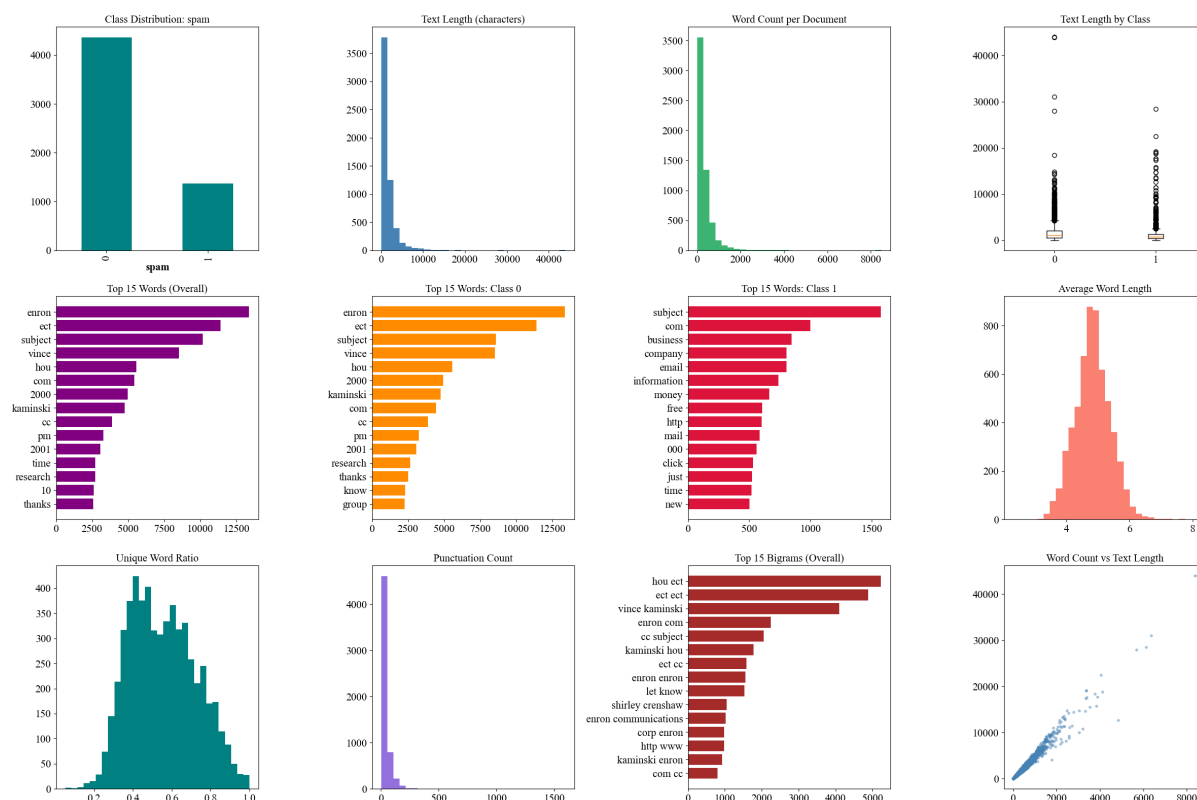


Figure 2: Generated 12-panel EDA figure – Email Spam Dataset (actual run output)

### Inference:

This one ran cleanly too. The top-words panels are the most telling part of this figure – "enron", "ect", "vince", "hou", "kaminski" dominate the overall and non-spam word lists, which gives away that this is (or is derived from) the well-known **Enron email corpus** rather than a generic email collection. The Class 1 (spam) word list looks completely different and much more like what you'd expect from marketing/phishing mail – "business", "money", "free", "click" – a nice sanity check that the labels are meaningful. The length-by-class boxplot and the word-count-vs-length scatter both show a small number of very long outlier emails (up to 40000+ characters) stretching the axis, so a real preprocessing step for this dataset would probably want to cap or log-transform text length before feeding it into a model.

## 5 Model Workflow Smoke Test – Iris and Diabetes

### Code Summary

Two generic harnesses – `run_classification_models()` and `run_regression_models()` – each loop over a dictionary of off-the-shelf Scikit-learn models, fit on a given train split, predict on the test split, and print standard metrics via matching `compute_*_metrics()` helper functions. These were smoke-tested using Scikit-learn’s **built-in toy datasets** (`load_iris`, `load_diabetes`) rather than a separately downloaded dataset – so this section validates that the harness code itself works correctly, using Iris in place of the manual’s “Iris Dataset” entry, and sklearn’s toy Diabetes regression set as a stand-in for (not the same as) the manual’s “Predicting Diabetes” Kaggle dataset.

### Classification Smoke Test – Iris (75/25 split, `random_state=42`)

| Model               | Accuracy / F1   |
|---------------------|-----------------|
| Logistic Regression | 1.0000 / 1.0000 |
| Decision Tree       | 1.0000 / 1.0000 |
| Random Forest       | 1.0000 / 1.0000 |
| SVM                 | 1.0000 / 1.0000 |
| KNN                 | 1.0000 / 1.0000 |
| Naive Bayes         | 1.0000 / 1.0000 |

#### Inference:

Every single model got a perfect score on the 38-sample test split (confusion matrix `[[15,0,0],[0,11,0],[0,0,12]]`, i.e. literally zero mistakes for every model). This isn’t a bug – Iris genuinely is close to linearly separable and is a famously “easy” benchmark dataset, so a perfect score here doesn’t tell us much about which model is “best”; it mainly confirms the harness code itself (train/predict/metric loop) is wired up correctly. A harder dataset would be needed to actually differentiate these 6 models.

### Regression Smoke Test – Diabetes (sklearn toy dataset, 75/25 split)

| Model                    | MAE   | RMSE  | R <sup>2</sup> |
|--------------------------|-------|-------|----------------|
| <b>Linear Regression</b> | 41.55 | 53.37 | <b>0.4849</b>  |
| Random Forest            | 43.46 | 54.57 | 0.4614         |
| KNN                      | 42.34 | 55.32 | 0.4466         |
| Ridge                    | 45.91 | 55.73 | 0.4384         |
| Lasso                    | 49.67 | 58.59 | 0.3791         |
| SVR                      | 57.01 | 67.17 | 0.1841         |
| Decision Tree            | 59.77 | 78.46 | −0.1132        |

#### Inference:

Plain Linear Regression came out on top with default hyperparameters, ahead of every more “sophisticated” model tried – a nice reminder that this small (442-sample, 10-feature), already-standardized toy dataset doesn’t have much non-linear structure for a tree or kernel method to exploit. Decision Tree’s **negative** R<sup>2</sup> (−0.113) is the most

notable result – worse than just predicting the mean every time – which is the text-book signature of an unpruned single tree overfitting badly on a small dataset with no `max_depth` constraint set.

## 6 ML Task Identification (for what was actually run)

| Dataset                | Type of ML Task                                                                                                                   | Feature Selection Technique                                                                                                   | Suitable ML Algorithm                                                                                                      |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Iris (sklearn toy)     | Classification (3-class, supervised)                                                                                              | Not applied in this run (all 4 features used as-is; well-separated so no selection needed in practice)                        | Any of the 6 tested – Logistic Regression is the simplest sufficient choice                                                |
| Loan Approval          | Classification (Approved/Rejected) – <i>note: as actually coded, not regression on amount as the dataset's usual name implies</i> | Not applied in this run                                                                                                       | Random Forest / Gradient Boosting (handles mixed numeric scales and the asset-value correlations seen in the heatmap well) |
| Email Spam             | Classification (binary, text/NLP)                                                                                                 | Not applied in this run (would typically use Chi-square or mutual information on the <code>CountVectorizer</code> vocabulary) | Naive Bayes or Logistic Regression on TF-IDF/count features                                                                |
| Diabetes (sklearn toy) | Regression                                                                                                                        | Not applied in this run                                                                                                       | Linear Regression / Ridge – the smoke-test results above directly support this                                             |

## 7 Key Learning Outcomes

- Writing one *generic* EDA function per data modality (tabular, text) is a genuinely efficient way to explore several datasets quickly, but genericness has a real cost – the “just grab the first two numeric columns” shortcut in the tabular EDA function silently picked a meaningless `loan_id` column here, which wouldve gone unnoticed without actually reading the output panels closely.
- The text-EDA function's top-word and bigram panels turned out to be a good free

sanity check on label quality – the spam-class vocabulary (“business”, “money”, “free”, “click”) looking genuinely different from the non-spam vocabulary (“enron”, “vince”, “hou”) is a quick way to confirm the labels are meaningful before ever training a model.

- Iris giving every model a perfect score, and Diabetes giving Linear Regression the win, is a good reminder that a model comparison is only as informative as the dataset is challenging – neither smoke test here should be read as “Logistic Regression/Linear Regression is the best model”, just that the harness code works.
- Matplotlib’s `rcParams` can be set globally once (as done here with a serif font and bold axis labels) to keep every figure in a project visually consistent without repeating style code in every function.
- Confirmed hands-on that the “type of ML task” isn’t always obvious from a dataset’s name alone – the Loan dataset’s actual target column made it a classification problem in practice, even though “Loan Amount Prediction” reads like a regression task.

## 8 Conclusion

Of the datasets explored, both the numeric (Loan Approval) and text (Email Spam) EDA pipelines ran successfully and produced genuinely informative 12-panel diagnostic figures, surfacing real characteristics of each dataset (asset-value correlations and a moderate class imbalance for Loan; an Enron-corpus fingerprint and heavy-tailed text lengths for Spam). The generic classification/regression harness was validated with Scikit-learn’s built-in Iris and Diabetes toy datasets, confirming the training/evaluation loop works correctly, though neither smoke test used a genuinely difficult dataset. Overall, this experiment achieved its core goal of hands-on familiarity with NumPy, Pandas, SciPy, Scikit-learn and Matplotlib across two different data modalities, while also surfacing one concrete, fixable issue (the blind first-two-columns heuristic in the tabular EDA function) that a more polished version of this toolkit would need to address.